

```
1 #Q1 Reverse a string and reverse the order of words Strings, loops
2 #Print the string reversed character by character.
3 text = "data engineering is fun"
4 print(text[::-1])
5
6 #Print the same string with the word order reversed, words themselves unchanged.
7 text = "data engineering is fun"
8 words = text.split()
9 reversed_words = " ".join(words[::-1])
10 print(reversed_words)
11
12 #Do the character reversal in two ways: using slicing and using a loop.
13 text = "data engineering is fun"
14 reversed_text = text[::-1]
15 print(reversed_text)
16
17
18 text = "data engineering is fun"
19 reversed_text = ""
20 for char in text:
21     reversed_text = char + reversed_text
22 print(reversed_text)
23
24
25
26
27
28
```

Run

script x



```
C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\ashud\PycharmProjects\WelcomeScreen\script.py
↑ nuf si gnireenigne atad
↓ fun is engineering data
⇌ nuf si gnireenigne atad
⇌ nuf si gnireenigne atad
Process finished with exit code 0
```

#Q2 Count vowels, consonants, digits and spaces in a string Strings, loops

```
text = "Data Engineer 2024 works on ETL pipelines"
```

```
vowels = 0
```

```
consonants = 0
```

```
digits = 0
```

```
spaces = 0
```

```
for char in text:
```

```
    char = char.lower()
```

```
    if char in "aeiou":
```

```
        vowels += 1
```

```
    elif char == " ":
```

```
        spaces += 1
```

```
    elif char.isalpha():
```

```
        consonants += 1
```

```
    elif char.isdigit():
```

```
        digits += 1
```

```
result = {
```

```
    "vowels": vowels,
```

```
    "consonants": consonants,
```

```
    "digits": digits,
```

```
    "spaces": spaces
```

```
}
```

```
print(result)
```

 script1 ×

:

```
C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\ashud\AppData\Local\Temp\pycharm-script1.py
{'vowels': 13, 'consonants': 18, 'digits': 4, 'spaces': 6}
```

```
Process finished with exit code 0
```

#Q3 Three classic loop questions: palindrome, factorial, Fibonacci

```
def is_palindrome(value): 2 usages
    value = str(value)
    return value == value[::-1]
```

```
def factorial_loop(n : {__add__}): 1 usage
    result = 1
```

```
    for i in range(1, n + 1):
        result *= i

    return result
```

```
def factorial_recursive(n : {__eq__, __mul__, __sub__}): 2 usages
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursive(n - 1)
```

```
def fibonacci(terms): 1 usage
    result = []
    a, b = 0, 1

    for i in range(terms):
        result.append(a)
        a, b = b, a + b

    return result
```

```
print(is_palindrome("madam"))
print(is_palindrome(121))
print(factorial_loop(5))
print(factorial_recursive(5))
print(fibonacci(8))
```

Run Question 3 x



```
↑ C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe "C:\Users\ashud\Py
↓ True
|| True
⇐ 120
⇐ 120
⇐ [0, 1, 1, 2, 3, 5, 8, 13]
⇐
Process finished with exit code 0
```

```
#Q4 Check whether two strings are anagrams
#once with sorted()
def is_anagram_sorted(a: {lower}, b: {lower}): 2 usages
    a = a.lower()
    b = b.lower()

    return sorted(a) == sorted(b)

a = "Listen"
b = "Silent"
c = "Data"
d = "Date"

print(is_anagram_sorted(a, b))
print(is_anagram_sorted(c, d))
```

```
#once with a frequency dictionary built in a loop.
def is_anagram_dict(a: {lower}, b: {lower}): 2 usages
    a = a.lower().replace(" ", "")
    b = b.lower().replace(" ", "")

    if len(a) != len(b):
        return False

    freq = {}

    for char in a:
        if char in freq:
            freq[char] += 1
        else:
            freq[char] = 1

    for char in b:
        if char in freq:
            freq[char] -= 1
        else:
            return False

    for count in freq.values():
        if count != 0:
            return False

    return True

a = "Listen"
b = "Silent"
c = "Data"
d = "Date"

print(is_anagram_dict(a, b))
print(is_anagram_dict(c, d))
```

Question4

```
C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Script
True
False
True
False
```

```

1 #Q5 Word frequency count and the three most frequent words
2 from collections import Counter
3 text = "data pipeline data quality pipeline data engineer quality pipeline data"
4 words = text.split()
5 freq = {}
6 for word in words:
7     if word in freq:
8         freq[word] += 1
9     else:
10        freq[word] = 1
11 print("Word frequency:", freq)
12 sorted_words = sorted(freq.items(), key=lambda x : x[1], reverse=True)
13 print("Top 3 words:")
14 for word, count in sorted_words[:3]:
15     print(word, "->", count)
16
17
18 freq = Counter(words)
19
20 print("Word frequency:", dict(freq))
21
22 print("Top 3 words:")
23 for word, count in freq.most_common(3):
24     print(word, "->", count)

```

Run Question5 ×



```

C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\ashud\Py
Word frequency: {'data': 4, 'pipeline': 3, 'quality': 2, 'engineer': 1}
Top 3 words:
data -> 4
pipeline -> 3
quality -> 2
Word frequency: {'data': 4, 'pipeline': 3, 'quality': 2, 'engineer': 1}
Top 3 words:
data -> 4
pipeline -> 3
quality -> 2

Process finished with exit code 0

```

```
#Q6 Remove duplicates from a list, and list the duplicated values
items = ['pune', 'mumbai', 'delhi', 'pune', 'chennai', 'mumbai', 'pune']
unique_items = set(str)(items)
print(unique_items)

seen = set()
unique_items = []

for item in items:
    if item not in seen:
        seen.add(item)
        unique_items.append(item)

print(unique_items)

frequency = {}
for item in items:
    if item in frequency:
        frequency[item] += 1
    else:
        frequency[item] = 1

for item, count in frequency.items():
    if count > 1:
        print(item, count)
```

n

Question 6 ×

■ | ⋮

```
C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe "C:\Us
{'mumbai', 'delhi', 'chennai', 'pune'}
['pune', 'mumbai', 'delhi', 'chennai']
pune 3
mumbai 2

Process finished with exit code 0
```

```
#Q7 Second largest number in a list, without using sort()
nums = [45, 88, 12, 88, 67, 23, 90, 90, 5]

largest = None
second_largest = None

for num in nums:
    # Skip duplicate of the largest number
    if num == largest:
        continue

    if largest is None or num > largest:
        second_largest = largest
        largest = num

    elif second_largest is None or num > second_largest:
        second_largest = num

print("Largest:", largest)
print("Second largest:", second_largest)
```

 Question 7 ×

⋮

C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe

Largest: 90

Second largest: 88

Process finished with exit code 0

```
#Q8 Compare two lists of files using set operations
source_files = ['cust_01.csv', 'cust_02.csv', 'cust_03.csv', 'cust_04.csv',
                'cust_02.csv']
target_files = [ 'cust_02.csv', 'cust_03.csv', 'cust_05.csv']

source = set [str] (source_files)
target = set [str] (target_files)

common = source & target

missing_in_target = source - target

extra_in_target = target - source

symmetric_difference = source ^ target

print("Common:", common)
print("Missing in target:", missing_in_target)
print("Extra in target:", extra_in_target)
print("Symmetric difference:", symmetric_difference)
```

Question8 x

```
C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\as
Common: {'cust_03.csv', 'cust_02.csv'}
Missing in target: {'cust_04.csv', 'cust_01.csv'}
Extra in target: {'cust_05.csv'}
Symmetric difference: {'cust_04.csv', 'cust_01.csv', 'cust_05.csv'}

Process finished with exit code 0
```



```
# Q9 Flatten a nested list
data = [1, [2, 3], [4, [5, 6]], 7, [[8, 9], 10]]
def flatten(data : {__iter__}): 2 usages
    result = []
    for item in data:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result
print(flatten(data))
```

 Question9 ×

```
C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\p
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[1, 2, 3, 4, [5, 6], 7, [8, 9], 10]

Process finished with exit code 0
```

```
#Q10 Clean a delimited string into a list and rebuild it as a string
```

```
line = " amit , neha , , raj , , simran , neha "
```

```
values = line.split(",")
```

```
cleaned = []
```

```
for value in values:
```

```
    value = value.strip()
```

```
    if value != "":
```

```
        cleaned.append(value)
```

```
unique_values = []
```

```
for value in cleaned:
```

```
    if value not in unique_values:
```

```
        unique_values.append(value)
```

```
rebuilt = "|".join(unique_values)
```

```
print("Cleaned list:", unique_values)
```

```
print("Rebuilt string:", rebuilt)
```

```
title_case = ", ".join(value.title() for value in unique_values)
```

```
print("Title Case:", title_case)
```

Question10 x

⏏

C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\ashud\Py

Cleaned list: ['amit', 'neha', 'raj', 'simran']

Rebuilt string: amit|neha|raj|simran

Title Case: Amit, Neha, Raj, Simran

Process finished with exit code 0

```

#Q11 Convert a list of strings to numbers, skipping the bad values
values = ['10', '25', 'abc', '30', '', '45.5', 'NULL', '60']
numbers = []
bad_values = []
for value in values:
    try:
        numbers.append(float(value))
    except ValueError:
        bad_values.append(value)
total = sum(numbers)
average = total / len(numbers)
print("Converted numbers:", numbers)
print("Rejected values:", bad_values)
print("Good values count:", len(numbers))
print("Bad values count:", len(bad_values))
print("Sum:", total)
print("Average:", average)

for value in values:
    value = value.strip()
    if value == "" or value.upper() == "NULL":
        bad_values.append(value)
        continue
    try:
        number = float(value)
        numbers.append(number)
    except ValueError:
        bad_values.append(value)
print(numbers)

```

Question11 ×

:

```

C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\U
Converted numbers: [10.0, 25.0, 30.0, 45.5, 60.0]
Rejected values: ['abc', '', 'NULL']
Good values count: 5
Bad values count: 3
Sum: 170.5
Average: 34.1
[10.0, 25.0, 30.0, 45.5, 60.0, 10.0, 25.0, 30.0, 45.5, 60.0]

```

```
#Q12 Split a list into batches of N
ids = [101, 102, 103, 104, 105, 106, 107, 108, 109, 110]
batch_size = 3

def chunk(data : {__len__, __getitem__}, size : int = 100): 1 usage
    if size <= 0:
        raise ValueError("Batch size must be greater than 0")

    batches = []

    for i in range(0, len(data), size):
        batches.append(data[i:i + size])

    return batches
```



```
batches = chunk(ids, batch_size)
print(batches)
for batch_number, batch in enumerate(batches, start=1):
    print("Batch", batch_number, ":", batch)
```

 Question12 x

⋮

```
C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\ashud\Py
[[101, 102, 103], [104, 105, 106], [107, 108, 109], [110]]
Batch 1 : [101, 102, 103]
Batch 2 : [104, 105, 106]
Batch 3 : [107, 108, 109]
Batch 4 : [110]

Process finished with exit code 0
```

#Q13 Merge two dictionaries and add the values of common keys

```
jan_sales = {  
    'East': 1200,  
    'West': 900,  
    'North': 1500  
}
```

```
feb_sales = {  
    'East': 800,  
    'South': 1100,  
    'North': 400  
}
```

```
combined_sales = {}
```

```
for region, sales in jan_sales.items():  
    combined_sales[region] = sales
```

```
for region, sales in feb_sales.items():  
    if region in combined_sales:  
        combined_sales[region] += sales  
    else:  
        combined_sales[region] = sales
```

```
print(combined_sales)
```

```
top_region = max(combined_sales, key=combined_sales.get)
```

```
print("Top region:", top_region)
```

```
print("Top sales:", combined_sales[top_region])
```

 Question13 x

| :

C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\ashud\Py

{'East': 2000, 'West': 900, 'North': 1900, 'South': 1100}

Top region: East

Top sales: 2000

Process finished with exit code 0

```

#Q14 Sort a dictionary by value, and sort a list of dictionaries by two keys
scores = {'amit': 78, 'neha': 92, 'raj': 65, 'simran': 92, 'kavya': 81}
employees = [
    {'name': 'Amit', 'dept': 'IT', 'salary': 50000},
    {'name': 'Neha', 'dept': 'HR', 'salary': 60000},
    {'name': 'Raj', 'dept': 'IT', 'salary': 65000},
    {'name': 'Simran', 'dept': 'Finance', 'salary': 60000},
]
sorted_scores = dict(sorted(scores.items(), key=lambda x: x[1], reverse=True))
top_3 = dict(sorted(sorted_scores.items(), key=lambda x: x[1], reverse=True)[:3])
print("Top 3 scores:", top_3)

sorted_employees = sorted(
    employees,
    key=lambda x: (x['dept'], -x['salary'])
)

print("Sorted employees:")

for employee in sorted_employees:
    print(employee)

highest_paid = {}

for employee in sorted_employees:
    dept = employee['dept']

    if dept not in highest_paid:
        highest_paid[dept] = employee

print("Highest paid by department:")

for dept, employee in highest_paid.items():
    print(dept, ":", employee)

```

Question14

```

C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\
Top 3 scores: {'neha': 92, 'simran': 92, 'kavya': 81}
Sorted employees:
{'name': 'Simran', 'dept': 'Finance', 'salary': 60000}
{'name': 'Neha', 'dept': 'HR', 'salary': 60000}
{'name': 'Raj', 'dept': 'IT', 'salary': 65000}
{'name': 'Amit', 'dept': 'IT', 'salary': 50000}
Highest paid by department:
Finance : {'name': 'Simran', 'dept': 'Finance', 'salary': 60000}
HR : {'name': 'Neha', 'dept': 'HR', 'salary': 60000}
IT : {'name': 'Raj', 'dept': 'IT', 'salary': 65000}

Process finished with exit code 0

```

```
#Q15 Group a list of records by a key - GROUP BY in plain Python
sales = [ ('East', 'Electronics', 25000), ('West', 'Apparel', 3200),
          ('East', 'Apparel', 4100), ('North', 'Electronics', 18000),
          ('East', 'Electronics', 32000), ('West', 'Grocery', 2200),]
```

```
grouped = {}
```

```
region_total = {}
```

```
region_orders = {}
```

```
region_category = {}
```

```
for region, category, amount in sales:
```

```
    if region not in grouped:
```

```
        grouped[region] = []
```

```
    grouped[region].append(amount)
```

```
    if region not in region_total:
```

```
        region_total[region] = 0
```

```
        region_orders[region] = 0
```

```
    region_total[region] += amount
```

```
    region_orders[region] += 1
```

```
    if region not in region_category:
```

```
        region_category[region] = {}
```

```
    if category not in region_category[region]:
```

```
        region_category[region][category] = 0
```

```
    region_category[region][category] += amount
```

```
print("Region -> amounts:", grouped)
```

```
print("Region -> total:", region_total)
```

```
print("Region -> orders:", region_orders)
```

```
print("Region -> category -> total:", region_category)
```

```
top_region = max(region_total, key=region_total.get)
```

```
print("Highest total region:", top_region)
```

```
print("Highest total:", region_total[top_region])
```

Activate Windows

Go to Settings to activate Windows.

```
C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\ashud\PycharmProjects\WelcomeScreen\Question15.py
```

```
Region -> amounts: {'East': [25000, 4100, 32000], 'West': [3200, 2200], 'North': [18000]}
```

```
Region -> total: {'East': 61100, 'West': 5400, 'North': 18000}
```

```
Region -> orders: {'East': 3, 'West': 2, 'North': 1}
```

```
Region -> category -> total: {'East': {'Electronics': 57000, 'Apparel': 4100}, 'West': {'Apparel': 3200, 'Grocery': 2200}, 'North': {'Electronics': 18000}}
```

```
Highest total region: East
```

```
Highest total: 61100
```

```
Process finished with exit code 0
```

```

# Q16 Build dictionaries from parallel lists using zip
headers = ['emp_id', 'name', 'dept', 'salary']

rows = [
    [1, 'Amit', 'IT', 50000],
    [2, 'Neha', 'HR', 60000],
    [3, 'Simran', 'Finance', 70000],
]

employees = []

for row in rows:
    employee = dict(zip(tuple(str | int | str) (headers, row))
    employees.append(employee)

print("Employees:")
print(employees)

print("\nAbove 55000:")

for employee in employees:
    if employee['salary'] > 55000:
        print(employee['name'], "->", employee['salary'])

id_to_name = {}

for employee in employees:
    id_to_name[employee['emp_id']] = employee['name']

print("\nID -> Name:")
print(id_to_name)

```

C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\ashud\PycharmProjects\WelcomeScreen\Question16.py

Employees:

```
[{'emp_id': 1, 'name': 'Amit', 'dept': 'IT', 'salary': 50000}, {'emp_id': 2, 'name': 'Neha', 'dept': 'HR', 'salary': 60000}, {'emp_id': 3, 'name': 'Simran', 'dept': 'Finance', 'salary': 70000}]
```

Above 55000:

Neha -> 60000

Simran -> 70000

ID -> Name:

```
{1: 'Amit', 2: 'Neha', 3: 'Simran'}
```

Process finished with exit code 0


```
# Q17 Functions with default arguments, *args, **kwargs and lambda
prices = [1200, 450, 3000, 780, 2500]

mapped = [x * 0.90 for x in prices]

filtered = [x for x in prices if x > 1000]

total = sum(filtered)

print("Mapped:", mapped)
print("Filtered:", filtered)
print("Total:", total)
```

 Question17 ×

⋮

C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C

Mapped: [1080.0, 405.0, 2700.0, 702.0, 2250.0]

Filtered: [1200, 3000, 2500]

Total: 6700

Process finished with exit code 0

```

# Q17 Functions with default arguments, *args, **kwargs and lambda
def order_summary(*items, discount:int=0, **details): 1 usage
    total = sum(items)
    discounted_total = total - (total * discount / 100)

    return {
        "total": total,
        "discounted_total": discounted_total,
        "details": details
    }

prices = [1200, 450, 3000, 780, 2500]

summary = order_summary(
    *items: *prices,
    discount=10,
    customer="Amit",
    payment="UPI",
    city="Delhi"
)
print("Summary:", summary)

discounted_prices = list[_S](
    map[float | int | Any](lambda price: {_mul_}: price * 0.90, prices)
)

print("Mapped:", discounted_prices)

filtered_prices = list[_T](
    filter(lambda price: {_gt_}: price > 1000, prices)
)

print("Filtered:", filtered_prices)

from functools import reduce

total = reduce(
    lambda a: {_add_}, b: a + b,
    filtered_prices
)

print("Total:", total)

prices = [1200, 450, 3000, 780, 2500]

mapped = [x * 0.90 for x in prices]

filtered = [x for x in prices if x > 1000]

total = sum(filtered)

print("Mapped:", mapped)
print("Filtered:", filtered)
print("Total:", total)

# For normal Python code, I'd generally prefer list comprehensions + sum():
# because they're usually:
# 1.easier to read
# 2.more Pythonic
# 3.easier to debug
# 4.less verbose than map()/filter()
# 5.sum() clearly communicates that we're calculating a total

```

```

C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\ashud\PycharmProjects\WelcomeScreen\Question17.py
Summary: {'total': 7930, 'discounted_total': 7137.0, 'details': {'customer': 'Amit', 'payment': 'UPI', 'city': 'Delhi'}}
Mapped: [1080.0, 405.0, 2700.0, 702.0, 2250.0]
Filtered: [1200, 3000, 2500]
Total: 6700
Mapped: [1080.0, 405.0, 2700.0, 702.0, 2250.0]
Filtered: [1200, 3000, 2500]
Total: 6700

```

Process finished with exit code 0

```
#Q18 Read a text file and report line, word and character counts
```

```
from collections import Counter
```

```
line_count = 0
```

```
word_count = 0
```

```
char_count = 0
```

```
longest_line = ""
```

```
data_line_count = 0
```

```
word_frequency = Counter()
```

```
with open("notes.txt", "r") as file:
```

```
    for line in file:
```

```
        line_count += 1
```

```
        clean_line = line.rstrip("\n")
```

```
        char_count += len(line)
```

```
        words = clean_line.split()
```

```
        word_count += len(words)
```

```
        if len(clean_line) > len(longest_line):
```

```
            longest_line = clean_line
```

```
        if "data" in clean_line.lower():
```

```
            data_line_count += 1
```

```
        word_frequency.update(word.lower() for word in words)
```

```
print("Lines:", line_count)
```

```
print("Words:", word_count)
```

```
print("Characters:", char_count)
```

```
print("Longest line:", longest_line)
```

```
print("Lines containing 'data':", data_line_count)
```

```
print("Top 3 words:", word_frequency.most_common(3))
```

```
# I would avoid read() because it loads the entire file into memory. I'd use a context manager and iterate over the file line by line.  
# This allows me to process the file incrementally with memory usage that doesn't grow with the total file size. I'd maintain only the  
# counters, the longest line, and the word-frequency data needed for the result.
```

Run

Question18 x

⏮ ■ ⏭ :

↑ C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\ashud\PycharmProjects\WelcomeScreen\Question18.py

↓ Lines: 0

⏮ Words: 0

⏭ Characters: 0

⏮ Longest line: |

⏭ Lines containing 'data': 0

⏮ Top 3 words: []

⏭ Process finished with exit code 0

```

1 #Q19 Read a CSV without pandas and aggregate it
2 import csv
3 from collections import defaultdict
4 from datetime import datetime
5
6 records = []
7
8
9 with open("sales.csv", "r", newline="") as file:
10     reader = csv.DictReader([str](file)
11
12     for row in reader:
13         row["amount"] = float(row["amount"])
14         records.append(row)
15
16 print("Records:")
17 print(records)
18
19
20 region_total = defaultdict[Any, float | int](float)
21 region_orders = defaultdict[Any, int](int)
22
23 for row in records:
24     region = row["region"]
25     amount = row["amount"]
26
27     region_total[region] += amount
28     region_orders[region] += 1
29
30
31 print("\nRegion-wise totals and averages:")
32
33 for region in region_total:
34     average = region_total[region] / region_orders[region]
35
36     print(
37         region,
38         "Total:", round(region_total[region], 2),
39         "Average:", round(average, 2)
40     )
41
42
43 top_region = max(region_total, key=region_total.get)
44
45 print("\nHighest total region:", top_region)
46 print("Highest total:", round(region_total[top_region], 2))
47
48
49 monthly_orders = defaultdict[Any, int](int)
50
51 for row in records:
52     date = datetime.strptime(row["order_date"], format="%Y-%m-%d")
53     month = date.strftime("%Y-%m")
54
55     monthly_orders[month] += 1
56
57 print("\nOrders per month:")
58
59 for month, count in monthly_orders.items():
60     print(month, ":", count)

```

```

C:\Users\ashud\PycharmProjects>WelcomeScreen\venv\Scripts\python.exe C:\Users\ashud\PycharmProjects>WelcomeScreen\Question19.py
Records:
[{'order_id': '1001', 'region': 'East', 'category': 'Electronics', 'amount': 25000.0, 'order_date': '2024-01-15'}, {'order_id': '1002', 'region': 'West', 'category': 'Apparel', 'amount': 3200.0, 'order_date': '2024-01-10'},

Region-wise totals and averages:
East Total: 61100.0 Average: 20366.67
West Total: 5400.0 Average: 2700.0
North Total: 23600.0 Average: 11800.0
South Total: 1500.0 Average: 1500.0

Highest total region: East
Highest total: 61100.0

Orders per month:
2024-01 : 2
2024-02 : 3
2024-03 : 3

Process finished with exit code 0

```

split(',') is acceptable for a guaranteed-simple file, but real CSV files can contain commas inside quoted fields, quotes, and other CSV-specific formatting. The csv module handles those cases correctly, so I'd use csv.DictReader for production data.

```

#Q20 Remove duplicate records from a file and write a clean file
import csv

duplicate_ids = set()
seen_ids = set()
records_by_id = {}

rows_read = 0

with open("customers.csv", "r", newline="") as file:
    reader = csv.DictReader(file)

    for row in reader:
        rows_read += 1

        cust_id = row["cust_id"]

        if cust_id not in records_by_id:
            records_by_id[cust_id] = row
            seen_ids.add(cust_id)

        else:
            duplicate_ids.add(cust_id)

exact_duplicate_ids = set()
conflicting_ids = set()

with open("customers.csv", "r", newline="") as file:
    reader = csv.DictReader(file)

    first_rows = {}

    for row in reader:
        cust_id = row["cust_id"]

        if cust_id not in first_rows:
            first_rows[cust_id] = row
        else:
            if row == first_rows[cust_id]:
                exact_duplicate_ids.add(cust_id)
            else:
                conflicting_ids.add(cust_id)

with open("customers_clean.csv", "w", newline="") as file:
    fieldnames = ["cust_id", "name", "city", "email"]

    writer = csv.DictWriter(file, fieldnames=fieldnames)

    writer.writeheader()

    for row in records_by_id.values():
        writer.writerow(row)

rows_written = len(records_by_id)
rows_dropped = rows_read - rows_written

print("Duplicate customer IDs:", duplicate_ids)
print("Exact duplicate IDs:", exact_duplicate_ids)
print("Conflicting IDs:", conflicting_ids)
print("Rows read:", rows_read)
print("Rows written:", rows_written)
print("Rows dropped:", rows_dropped)

```

Run Question20 x



```
C:\Users\ashud\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\ashud\PycharmProjects\WelcomeScreen\Question20.py
Duplicate customer IDs: {'101', '102', '103'}
Exact duplicate IDs: {'101', '103'}
Conflicting IDs: {'102'}
Rows read: 7
Rows written: 4
Rows dropped: 3

Process finished with exit code 0
|
```